Polybius Group Frame

Interpreter Cards, Gauge Pauses, Tap-Code Streams, and Sealed OMI Projection Channels

0. Status

This document defines the **Polybius Group Frame** for OMI.

It is not a plaintext cipher.

It is not a literal geometric honeycomb.

It is an interpreter-card system for cycling through the four canonical OMI projection channels:

```
FS
GS
RS
US
```

The frame lets OMI target groups of already-valid states by using:

```
Schläfli / Polybius row-column selection
canonical sealed gauge masks
Base36 socket references
Base64 payload carriers
Q_xy projection
slot5040 replay placement
receipt acceptance
```

The core doctrine remains:

```
Projection does not authorize state.
Receipt accepts state.
```

---

1. The Projection Spine

The OMI projection spine is palindromic:

```
[FS] [GS] [RS] [US@[CAR: base36][CDR: base64]]
                         [US@[CAR: base36][CDR: base64]] [RS] [GS] [FS]
  ↓    ↓    ↓                     ↓      ↓                       ↓    ↓    ↓
file group record            unit  point                    record group file
```

Expanded as a mirrored walk:

```
file
  group
    record
      unit
        point
      unit
    record
  group
file
```

This is the projection spine:

```
FS -> GS -> RS -> US -> US -> RS -> GS -> FS
```

Meaning:

Channel	Role	Projection
`FS`	file surface	object / file / carrier boundary
`GS`	group surface	path / group / context
`RS`	record surface	query / record / relation
`US`	unit surface	socket / unit / point

The `US` surface is split into a cons pair:

```
US@[CAR: base36][CDR: base64]
```

Meaning:

```
CAR = Base36 reference socket
CDR = Base64 payload carrier
```

So:

```
Base36 names the socket.
Base64 carries the bytes.
```

---

2. Tap-Code Analogy

The ordinary tap code uses a Polybius square.

A letter is encoded by two taps:

```
row taps
short pause
column taps
long pause
```

Example:

```
B = row 1, column 2
```

The listener distinguishes row, column, and letter boundary by timing.

OMI adapts this idea, but does not use it as an alphabet cipher.

OMI uses the same pattern as an interpreter-card protocol:

```
row rail
gauge pause
column rail
acceptance pause
```

In OMI:

Tap-code role	OMI role
row taps	`{p,∞}` rail
short pause	sealed gauge mask
column taps	`{∞,r}` rail

Tap-code role	OMI role
long pause	receipt / acceptance boundary

The gauge mask acts like the pause.

It tells the interpreter which card is active:

```
FS
GS
RS
US
```

The acceptance suffix tells the interpreter whether the projected channel is sealed:

```
0xAA55
```

Therefore:

```
tap timing becomes gauge selection
letter boundary becomes receipt boundary
```

---

3. The Delta Law as Stream Cycler

The interpreter cards cycle through a deterministic bitwise law:

```
Δ_C(x) = rotl(x,1) XOR rotl(x,3) XOR rotr(x,2) XOR C
```

Four choices define the law:

```
rotations, not shifts  -> no bits are lost
XOR                   -> reversible mixing
constant C            -> breaks the zero fixed point
mask to width         -> bounded state
```

The law does not invent meaning.

It cycles interpreter state.

The projection cards are selected by the lawful orbit.

Compact rule:

```
Delta cycles.
Gauge pauses.
Polybius selects.
Base36 indexes.
Base64 carries.
Q_xy projects.
Receipt accepts.
```

---

4. The Polybius Group Frame

The group frame is Schläfli-indexed:

```
{p,∞,r}
```

Where:

```
p = row-side finite selector
∞ = open relation / unbounded path rail
r = column-side finite selector
```

The frame is:

```
|  {p,∞,r} === {omi---imo} === {0x00-0x3f} === {[FS],[GS],[RS],[US]}    |
|     | {∞,r}|   {∞,2}|    {∞,3}|    {∞,4}|    {∞,5}|    {∞,6}|    {∞,∞}|
|-----|------|--------|---------|---------|---------|---------|---------|
|{p,∞}|   p\r|  0x0055|   0x0066|   0x0077|   0x0088|   0x0099|   0x00AA|
|-----|------|--------|---------|---------|---------|---------|---------|
|{2,∞}|0x5500|{2,∞,2} | {2,∞,3} | {2,∞,4} | {2,∞,5} | {2,∞,6} | {2,∞,∞} |
|{3,∞}|0x6600|{3,∞,2} | {3,∞,3} | {3,∞,4} | {3,∞,5} | {3,∞,6} | {3,∞,∞} |
|{4,∞}|0x7700|{4,∞,2} | {4,∞,3} | {4,∞,4} | {4,∞,5} | {4,∞,6} | {4,∞,∞} |
|{5,∞}|0x8800|{5,∞,2} | {5,∞,3} | {5,∞,4} | {5,∞,5} | {5,∞,6} | {5,∞,∞} |
|{6,∞}|0x9900|{6,∞,2} | {6,∞,3} | {6,∞,4} | {6,∞,5} | {6,∞,6} | {6,∞,∞} |
|{∞,∞}|0xAA00|{∞,∞,2} | {∞,∞,3} | {∞,∞,4} | {∞,∞,5} | {∞,∞,6} | {∞,∞,∞} |
```

The rail map is:

```
2 -> 0x55
3 -> 0x66
4 -> 0x77
5 -> 0x88
6 -> 0x99
∞ -> 0xAA
```

A paired selector is:

```
word16(p,r) = rail(p) << 8 | rail(r)
```

Examples:

```
word16(2,2) = 0x5555
word16(2,∞) = 0x55AA
word16(∞,2) = 0xAA55
word16(∞,∞) = 0xAAAA
```

Interpretation:

```
0x55 = finite rail generator
0xAA = infinity / sentinel rail
0xAA55 = infinity-to-finite acceptance bridge
0x55AA = finite-to-infinity mirror bridge
```

The frame is not literal geometry in the DOM.

It is a group-selection matrix for validated OMI states.

---

5. Sealed Gauge Words

The OMI gauge selector is sealed by the acceptance suffix:

```
sealedGauge = (gaugeMask << 16) | 0xAA55
```

Canonical gauge masks:

```
FS = 0x0001
GS = 0x0010
```

```
RS = 0x0100
US = 0x1000
```

Therefore:

```
0x0001AA55 gauge = o---o  = object / FS
0x0010AA55 gauge = /---/  = path / GS
0x0100AA55 gauge = ?---?  = query / RS
0x1000AA55 gauge = @---@  = socket / US
```

The `0xAA55` suffix is the acceptance bridge.

The gauge mask selects which projection channel is being sealed.

The sealed gauge word is the interpreter-card pause.

It tells the stream:

```
pause here
read this subgroup under this channel
continue the tap-code stream
```

---

6. The Four Interpreter Cards

The four gauges are interpreter cards.

FS Card

```
0x0001AA55
```

```
o---o
```

Role:

```
object
file
carrier boundary
local memory object reference
```

GS Card

```
0x0010AA55
```

```
/---/
```

Role:

```
path
group
context
relational descent
```

RS Card

```
0x0100AA55
```

```
?---?
```

Role:

```
query
record
payload claim plane
relation surface
```

US Card

```
0x1000AA55
```

```
@---@
```

Role:

```
socket
unit
point
```

```
Base36 CAR
Base64 CDR
```

The US card is the point/unit bridge:

```
US@[CAR: base36][CDR: base64]
```

Meaning:

```
CAR = where the payload is referenced
CDR = what payload is carried
```

---

7. Nibble-Walk

The four gauges map into the nibble walk:

```
0x0 gauge = o---o  = object / FS
0x1 gauge = /---/  = path / GS
0x2 gauge = ?---?  = query / RS
0x3 gauge = @---@  = socket / US
```

A 16-bit nibble-walk reads:

```
high nibble = gauge / row / region
low nibble  = slot / local selector
```

The Base36 socket resolves into a bounded local coordinate:

```
Base36 socket -> value36

region36 = floor(value36 / 16)
local16  = value36 mod 16

x = local16 mod 4
y = floor(local16 / 4)
```

Then:

```
x,y -> Q_xy
Q_xy -> local240
local240 -> slot5040
```

This keeps all Base36 positions.

The 4×4 local cell is not the whole Base36 alphabet.

It is the local cell inside a larger Base36 region.

---

8. Bridge Projection

The projection bridge is:

```
Q_xy(x,y) = 60x² + 16xy + 4y²
```

Factored:

```
Q_xy(x,y) = 4(15x² + 4xy + y²)
```

This exposes:

```
4  = selector factor
15 = active nibble
4  = selector cross-term
1  = identity / payload term
```

The local bridge values are:

```
local240 = Q_xy(x,y) mod 240
slot5040 = fano7×720 + role3×240 + local240
```

So the path is:

```
Base36 socket
-> local16
-> x,y
-> Q_xy
```

```
-> local240
-> slot5040
```

This makes the socket computationally addressable.

---

9. Tap-Code Stream of Subgroups

A normal tap-code stream separates row and column by a pause.

OMI separates subgroup interpretation by a sealed gauge word.

A subgroup stream may be read:

```
{p,∞}
sealedGauge
{∞,r}
US@[CAR][CDR]
receipt
```

The sealed gauge word is the pause that determines the interpreter card.

Example:

```
{2,∞}
0x0001AA55
{∞,2}
```

means:

```
read the {2,∞,2} group under the FS object card
```

Example:

```
{∞,2}
0x1000AA55
US@[CAR:3C][CDR:<base64url>]
```

means:

```
read socket 3C as a US unit/point carrier
```

The result is not accepted merely because the stream can be read.

It becomes accepted only after receipt.

---

10. DOM Projection Boundary

The DOM still needs only six projection handles:

```
id
data-omi
data-imo
<o>
<omi>
<imo>
```

The Polybius Group Frame does not add new DOM authority.

It only determines how OMI-Matrix targets groups of validated slices.

DOM example:

```
<o
  id="o---o/---/?v=...;l=...;h=...;b=beta1;s={4,3}@3C@"
  data-omi="o---o/---/?v=...;l=...;h=...;b=beta1;s={4,3}@3C@"
  data-imo="o---o/---/?receipt=accepted@3C@"
></o>
```

Here:

```
id       = selector hook
data-omi = readable declaration projection
data-imo = interpreted / receipt projection
<o>      = object carrier boundary
```

The DOM exposes.

The receipt accepts.

---

11. Slice Mode and Group Mode

OMI-Mirror is slice mode:

```
ψ(frame, path, socket, receipt) -> one projected state
```

OMI-Matrix is group mode:

```
matrix(frame, path pattern, socket region) -> accepted group of slices
```

The Polybius Group Frame belongs to OMI-Matrix.

It lets OMI-Matrix cycle interpreter cards and target groups:

```
FS group
GS group
RS group
US group
```

without making the DOM itself geometric.

---

12. Canon Statement

```
The Polybius Group Frame is the interpreter-card frame of OMI-Matrix.

It cycles through FS, GS, RS, and US by sealed gauge masks.

The gauge mask is the pause in the tap-code stream.

The Schläfli rails select the subgroup.

The Base36 CAR names the socket.

The Base64 CDR carries the payload.

The quadratic form projects the local coordinate.

The replay ring records the slot.

The DOM exposes the projection.
```

```
The receipt accepts the state.
```

Shortest form:

```
rail selects
gauge pauses
socket indexes
payload carries
quadratic projects
receipt accepts
```